

第六章：中间代码优化（2）

公共表达式节省

- ◆ 公共表达式：是指两个表达式中的子表达式他们的**计算结果相同**。

$a*b$ $4*5$

c/d $20/1$

$e+f$ $10+10$

是不是公共
表达式？

强化定义：子表达式运算符w相同，且左右分量**值**相同

- ◆ 公共表达式节省在局部进行的时候都是针对基本块的。

公共表达式节省

强化定义：子表达式运算符 w 相同，且左右分量值相同

(w_1, a, b, t_1)

(w_2, c, d, t_2)

若 $w_1=w_2$ ，且 $V(a)=V(c)$, $V(b)=V(d)$
则为公共表达式。

核心问题：如何找到公共子表达式

公共表达式节省

要抛开表面看本质!!!

- ◆ 设有下列语句：(e=b, f=c)

t := b * c ;

e := b * c + e * f ; 长的不一样其实一样

c := b * c + 10 ;

d := b * c + d ;

→ 长的一样其实不一样

- ◆ 优化后，可得到下列语句：

t := b * c ; e := t + t ;

c := t + 10 ; d := b * c + d ;

基于值编码的公共表达式节省

值编码优化方法的主要思想：

对中间代码中出现的每个值确定一个编码，使得具有相同值的运算分量对应的编码相同。

一个值 $\Rightarrow$ 一个编码

三个事：

1. 值编码的分配 $\Rightarrow$ 值编码表
2. 四元式的值编码表示 $\Rightarrow$ 四元式映象码
3. 优化后四元式的变化 $\Rightarrow$ 中间变量等价表

$(+, b, c, t) \Rightarrow (+, \textcircled{1}, \textcircled{2}, t)$

基于值编码的公共表达式节省

- ◆ 编码原理：
- ◆ 用到一张值编码表，表示分量当前值的编码
- ◆ 若当前考察的代码为 $dk: (\omega, u1, u2, u3)$ ：
 - 若值编码表中已有 (u_i, m_i) 则令 $dk: u_i$ 的值编码为 m_i , $i=1, 2, 3$
 - 否则为 $dk: u_i$ 创建一个新编码 m 填入编码表
- ◆ 若考察代码为 $dk: (: =, u1, -, u2)$ ：
 - $u1$ 的处理与之前相同
 - 令 $dk: u2$ 的编码与 $dk: u1$ 的编码相同

基于值编码的公共表达式节省

➤ 编码原理：

- ◆ 用到一张值编码表，表示分量当前值的编码

对于 (w, U_1, U_2, U_3)

- ◆ 若 U_i 的在表里已有，直接对该四元式中的 U_i 赋编码
- ◆ 若无，则创建，为 U_i 赋予一个新编码

对于 $(:=, U_1, _, U_2)$

- ◆ U_1 如上
- ◆ U_2 的编码与 U_1 相同

基于值编码的公共表达式节省

◆ 值编码性质

1. 相同常量一定具有相同值编码

2. 不同常量的编码值一定不同

对变量来说并非如此

1. 不同分量上的相同变量未必具有相同编码

2. 每当一个变量 x 被赋值， x 将得到一个新的编码，使得后面代码中的 x 分量取编码值 n ，直至 x 再被赋值

基于值编码的公共表达式节省

◆ 在分析的过程中对应每条四元式还要生成一个编码四元式。

◆ $\mu(x)$ 表示任意运算分量 x 的值编码；

◆ 把 $(\omega, \mu(A), \mu(B), \mu(t))$ 叫作 (ω, A, B, t) 的 **映象码**；

$(\omega, A, B, t) \Rightarrow (\omega, \mu(A), \mu(B), \mu(t))$
四元式各部分都用值编码表示（也就是看到本质了），映象码相同的四元式，就是公共表达式。

基于值编码的公共表达式节省

临时变量怎么办？？

➤ 临时变量等价表PAIR:

$(+, a, b, t_i)$

~~$(+, a, b, t_j)$~~

(t_i, t_j) 表示 $t_j = t_i$

当发生节省时， t_j 需要替换为 t_i 。

基于值编码的公共表达式节省

算法流程：从基本块的第一条四元式开始

- ◆ 等价变量表替换
- ◆ 对四元式中的分量进行编码
- ◆ 值编码表替换，生成编码四元式（当前表达式的映象码）
- ◆ 遇到运算型四元式，查当前基本块中已处理过表达式的映象码中是否存在与编码四元式运算符、运算分量都相同的，若存在则说明当前表达式为公共表达式，删去当前四元式，把等价的名字填入等价表
- ◆ 遇到赋值($:=$, a , $-$, b)， b 的编码赋值为 a 的编码

循环不变式外提

- ◆ 循环不变式: 如果一个表达式E的值在一个循环中不会被改变, 则称它为该循环的不变表达式。

强化定义: 表达式E左右分量的值在循环中不被改变。

- ◆ 循环不变式外提: 将循环不变式提到循环外面进行计算, 可减少循环的运行时间。

例如，假设有程序段：

```
i:=1; // t=x*y;
```

```
while i<=1000 do
```

```
    begin a[i]:= x*y //t ; i:=i+1 end;
```

则 $x*y$ 是该循环的不变表达式，可以把它提到循环外边

此种优化收益巨大，
是主要考察的优化
方式。

矩阵乘法：有a，b为10*10数组

```
for (i=1~10)
```

```
    for (j=1~10)
```

```
        for (k=1~10)
```

```
            c[i][j]=c[i][j]+a[i][k]*b[k][j];
```

a[i]地址的计算与j、k循环无关，c[i][j]地址的计算与k循环无关

循环不变式外提

- ◆ 同样也是针对**运算型表达式**（不对赋值等其他四元式外提）

a=1;

while (e) {a=2};

b=a+1 b=??

但凡有可能出错的优化，都不进行！！

循环不变式外提

思想：

1. 确定哪些变量是循环变量，既被循环改变了值的量；
2. 确定循环不变式，然后外提。
3. 为此需扫描循环体代码两遍。

第一遍：看哪些变量的值变了（建立定值表）

$(w, A, B, t) \rightarrow t$ 被定值了，记入LoopDef

$(:=, a, _, b) \rightarrow b$ 被定值了，记入LoopDef

第二遍：确定哪些没变（进行不变式外提）

(w, A, B, t) 若A，B都不在LoopDef表里，说明AwB是循环不变式，则可外提，并将t从LoopDef表里删去

循环不变式外提（单层循环）

- ◆ 将源程序转换成四元式序列；
- ◆ 划分基本块；
- ◆ 循环定值表LoopDef初始为空；
- ◆ 对循环体四元式进行**第一遍扫描（建表）**，把有定值的变量填到变量定值表中，若它是一个运算型四元式 $(\omega_1, A_1, B_1, t_1)$ ，则把 t_1 填到表中，若为赋值型四元式 $(:=, a, -, b)$ 则把 b 填入表中
- ◆ 循环不变式外提为**第二遍扫描（外提）**，每遇到一个运算型四元式 $(\omega_1, A_1, B_1, t_1)$ ，若 A_1 、 B_1 都不在变量定值表中，则将其提到循环体外，同时在变量定值表中删去 t_1

```

i:=1
while i<=100 do
  begin
    z:=i*k*5;
    a:=2*k+2*k*2;
    i:=i+1;
  end

```

就是四元式最
后一项

LoopDef

={t1, t2, t3, z,
t4, ~~t5~~, t6, t7, a, t8, i}

优化前四元式序列:

1. (ASSIG, 1, —, i)
2. (WHILE, —, —, —)
3. (LE, i, 100, t1)
4. (DO, t1, —, —)
5. (MULTI, i, k, t2)
6. (MULTI, t2, 5, t3)
7. (ASSIG, t3, —, z)
8. (MULTI, 2, k, t4)
9. (MULTI, 2, k, t5)
10. (MULTI, ~~t4~~, 2, t6)
11. (ADDI, t4, t6, t7)
12. (ASSIG, t7, —, a)
13. (ADDI, i, 1, t8)
14. (ASSIG, t8, —, i)
15. (ENDWHILE, —, —, —)

循环优化前四元式序列:

1. (ASSIG, 1, —, i)
2. (WHILE, —, —, —)
3.  (LE, i, 100, t1)
4. (DO, t1, —, —)
5. (MULTI, i, k, t2)
6. (MULTI, t2, 5, t3)
7. (ASSIG, t3, —, z)
8. (MULTI, 2, k, t4)
9. (MULTI, t4, 2, t6)
10. (ADDI, t4, t6, t7)
11. (ASSIG, t7, —, a)
12. (ADDI, i, 1, t8)
13. (ASSIG, t8, —, i)
14. (ENDWHILE, —, —, —)

LoopDef={t1, t2, t3, z,
t4, t6, t7, a, t8, i}



循环优化前四元式序列：

1. (ASSIG, 1, —, i)
2. (WHILE, -, -, -)
3. (LE, i, 100, t1)
4. (DO, t1, —, —)
5. (MULTI, i, k, t2)
6. (MULTI, t2, 5, t3)
7. (ASSIG, t3, —, z)
8. (MULTI, 2, k, t4)
9. (MULTI, t4, 2, t6)
10. (ADDI, t4, t6, t7)
11. (ASSIG, t7, —, a)
12. (ADDI, i, 1, t8)
13. (ASSIG, t8, —, i)
14. (ENDWHILE, -, -, -)

循环优化后四元式序列：

1. (ASSIG, 1, —, i)
2. (MULTI, 2, k, t4)
3. (MULTI, t4, 2, t6)
4. (ADDI, t4, t6, t7)
5. (WHILE, -, -, -)
6. (LE, i, 100, t1)
7. (DO, t1, —, —)
8. (MULTI, i, k, t2)
9. (MULTI, t2, 5, t3)
10. (ASSIG, t3, —, z)
11. (ASSIG, t7, —, a)
12. (ADDI, i, 1, t8)
13. (ASSIG, t8, —, i)
14. (ENDWHILE, -, -, -)

循环不变式外提中注意的问题

1. 多层循环问题：每层循环建一个LoopDef，里层变量定值表属于外层变量定值表；从最内层循环循环体开始外提，一个循环不变式(w, A, B, t)从里层开始可以被外提若干次，每提一次， t 从内层LoopDef删去，并填入外层LoopDef中。

2. 除法不外提 `While(y≠0) {...(/, a, y, t)...}`
可能会引起除0的情况

3. 赋值绝不外提 `a = 2; while (e) { a=10; }`

循环不变式外提中注意的问题

4.非良性循环（循环体有函数调用或地址引用型变量的赋值）不做外提优化

可能会有副作用： $a=f(b)$ 可能会访问静态变量、全局变量。

可能对变量进行了隐形赋值：

$a=\&b$ 这里没有对d显性赋值，我们的方法可以
 $*a=c$ 外提
 $d=b+1$

没法构造准确的变量定值表。

该情况不做循环不变式外提（整体都不做）！

循环不变式外提中注意的问题

5. 运算型四元式的运算分量是间接寻址时，该四元式不外提

只对该条语句不做外提。 $(w, a(\text{indir}), b, t)$

地址加运算可以外提

while (e) {	$(-, 3, 1, t_0)$	$(-, 3, 1, t_3)$
...	$(*, t_0, 1, t_1)$	$(*, t_3, 1, t_4)$
a[3]=a[3]+1	$([], a, t_1, t_2)$	$([], a, t_4, t_5)$
...}	$(+, t_5, 1, t_6)$	
	$(:=, t_2, _, t_6)$	

可外提？？？

例子

例2: 已知: $a: \text{array}[1..10][1..1000]$ of integer; 设有如下循环语句:

```
j:=1
while j<=1000 do
  begin a[i][j]:=0;
        j:=j+1;
  end;
```


优化前的四元式序列:

```
(ASSIG, 1, —, j)
(WHILE, —, —, —)
(LE, j, 1000, t1)
(DO, t1, —, —)
(SUBI, i, 1, t2)
(MULTI, t2, 1000, t3)
(AADD, a, t3, t4)
(SUBI, j, 1, t5)
(MULTI, t5, 1, t6)
(AADD, t4, t6, t7)
(ASSIG, 0, —, t7)
(ADDI, j, 1, t8)
(ASSIG, t8, —, j)
(ENDWHILE, —, —, —)
```

LoopDef={t1, t2, t3,
t4, t5, t6, t7, t8, j}

优化后的四元式序列:

```
(ASSIG, 1, —, j)
(SUBI, i, 1, t2)
(MULTI, t2, 1000, t3)
(AADD, a, t3, t4)
(WHILE, —, —, —)
(LE, j, 1000, t1)
(DO, t1, —, —)
(SUBI, j, 1, t5)
(MULTI, t5, 1, t6)
(AADD, t4, t6, t7)
(ASSIG, 0, —, t7)
(ADDI, j, 1, t8)
(ASSIG, t8, —, j)
(ENDWHILE, —, —, —)
```

LoopDef={t1, t5, t6, t7, t8, j}

```
j:=1
while j<=1000 do
    begin a[i][j]:=0;
        j:=j+1;
    end;
```

给出如下程序段的四元式中间代码，并利用常表达式节省、公共表达式节省和循环不变式外提三种优化技术对中间代码进行优化。其中A: array[1..100] of integer, 程序中的所有变量都为整型变量，占1个存储单元。

```
a:=0;
j:=(a+1)*5;
while j<100 do
begin
    a:=a+A[3];
    if a<10 then k:=0;
    else k:=x*y+x*y/5;
    j:=j+1;
end
```

中间代码:

1. ($:=$, 0, $_$, a)
2. (+, a, 1, t1)
3. (*, t1, 5, t2)
4. ($:=$, t2, $_$, j) **B1**

5. (WHILE, $_$, $_$, $_$)
6. (<, j, 100, t3)
7. (Do, t3, $_$, $_$) **B2**

8. (-, 3, 1, t4)
9. (*, t4, 1, t5)
10. (AADD, A, t5, t6)
11. (+, a, t6, t7)
12. ($:=$, t7, $_$, a)
13. (<, a, 10, t8) **B3**
14. (THEN, t8, $_$, $_$)

14. .

15. ($:=$, 0, $_$, k)
16. (ELSE, $_$, $_$, $_$) **B4**

17. (*, x, y, t9)
18. (*, x, y, t10)
19. (/ , t10, 5, t11)
20. (+, t9, t11, t12)
21. ($:=$, t12, $_$, k) **B5**

22. (ENDIF, $_$, $_$, $_$)
23. (+, j, 1, t13) **B6**
24. ($:=$, t13, $_$, j)
25. (ENDWHILE, $_$, $_$, $_$)

第一步：划分基本块

①先看常表达式节省

1. ($:=$, 0, $_$, a)
~~2. (+, a, 1, t1)~~
~~3. (*, t1, 5, t2)~~
4. ($:=$, t2, $_$, j) **B1**

5. (WHILE, $_$, $_$, $_$)
6. (<, j, 100, t3)
7. (Do, t3, $_$, $_$) **B2**

~~8. (-, 3, 1, t4)~~
~~9. (*, t4, 1, t5)~~
10. (AADD, A, t5, t6)
11. (+, a, t6, t7)
12. ($:=$, t7, $_$, a)
13. (<, a, 10, t8) **B3**
14. (THEN, t8, $_$, $_$)

常量表

(a, 0)
(t1, 1)
(t2, 5)
(t4, 2)
(t5, 2)

后面部分没有常表达式节省了

②公共表达式节省

14. .

15. (:=, 0, _, k)

16. (ELSE, _, _, _) **B4**

17. (*, x, y, t9)

~~18. (*, x, y, t10)~~

19. (/ ~~t9~~, 5, t11)

20. (+, t9, t11, t12)

21. (:=, t12, _, k) **B5**

22. (ENDIF, _, _, _)

23. (+, j, 1, t13) **B6**

24. (:=, t13, _, j)

25. (ENDWHILE, _, _, _)

值编码表 映象表 等价表

(x, 1)
(y, 2)
(t9, 3)
(5, 4)
(t ₁₁ , 5)
(t ₁₂ , 6)
(k, 7)

(*, 1, 2, t10) (t10, t9)

③循环不变式外提

1. ($:=$, 0, $_$, a)
2. (+, a, 1, t1)
3. (*, t1, 5, t2)
4. ($:=$, t2, $_$, j) **B1**

5. (WHILE, $_$, $_$, $_$)
6. (<, j, 100, t3)
7. (Do, t3, $_$, $_$) **B2**

~~8. (-, 3, 1, t4)~~
~~9. (*, t4, 1, t5)~~
10. (AADD, A, t5, t6)
11. (+, a, t6, t7)
12. ($:=$, t7, $_$, a)
13. (<, a, 10, t8) **B3**
14. (THEN, t8, $_$, $_$)

14. .

15. ($:=$, 0, $_$, k)
16. (ELSE, $_$, $_$, $_$) **B4**

17. (*, x, y, t9)
~~18. (*, x, y, t10)~~
19. (/ , t9, 5, t11) **除法不外提**
20. (+, t9, t11, t12)
21. ($:=$, t12, $_$, k) **B5**

22. (ENDIF, $_$, $_$, $_$)
23. (+, j, 1, t13) **B6**
24. ($:=$, t13, $_$, j)
25. (ENDWHILE, $_$, $_$, $_$)

10外提
定值表Loopdef (t3, ~~t6~~, t7, a, t8, k, ~~t9~~, t11, t12, k, t13, j)

17外提

优化后的中间代码:

1. ($:=$, 0, $_$, a)
2. ($:=$, 5, $_$, j)
3. (AADD, A, 2, t6)
4. ($*$, x, y, t9)
5. (WHILE, $_$, $_$, $_$)
6. ($<$, j, 100, t3)
7. (Do, t3, $_$, $_$)
8. ($+$, a, t6, t7)
9. ($:=$, t7, $_$, a)
10. ($<$, a, 10, t8)
11. (THEN, t8, $_$, $_$)
12. ($:=$, 0, $_$, k)
13. (ELSE, $_$, $_$, $_$)
14. ($/$, t9, 5, t11)
15. ($+$, t9, t11, t12)
16. ($:=$, t12, $_$, k)
17. (ENDIF, $_$, $_$, $_$)
18. ($+$, j, 1, t13)
19. ($:=$, t13, $_$, j)
20. (ENDWHILE, $_$, $_$, $_$)

例: A:array[1...100] integer;

if (j==1) 可节省

then A[j]=A[j]+1

else A[j]=j+1

可节省么?? 不可以